

PROGRAMMIERUNG I

Kapitel 2: Methoden, Klassen und Objekte

Methoden

Komplexe Software besteht häufig aus mehreren tausend Zeilen Quellcode (z.B. *Linux-Kernel*: über 15 Millionen Zeilen). Hier ist eine gute Struktur des Programmcodes unerlässlich. Diese Strukturierung kann auf unterschiedliche Weisen erreicht werden, wobei die grundlegendste aller Möglichkeit **Funktionen** sind, bei Java **Methoden** genannt.

Aufbau einer Methode

Methodensignatur

Visibilität	Definiert, von wo aus diese Methode aufgerufen werden kann. Wird diese Angabe weg gelassen, wird die Default-Visibilität verwendet. Visibilität wird ausführlich in Kapitel 3 behandelt.
Return-Type	Eine Methode kann mittels return maximal einen Wert zurückliefern, der dem in der Signatur angegebenen Typen entsprechen muss. Soll kein Wert zurück gegeben werden, ist der Return-Type void anzugeben. Dementsprechend darf dann im Methodenrumpf kein Wert mittels return zurückgeliefert werden.
Name	Für Methoden-Namen gelten die gleichen Konventionen wie für Variablen
Parameter	Beim Aufruf der Methode können beliebig viele Parameter übergeben werden, die dann innerhalb des Methodenrumpfes verwendet werden können.

Syntax:

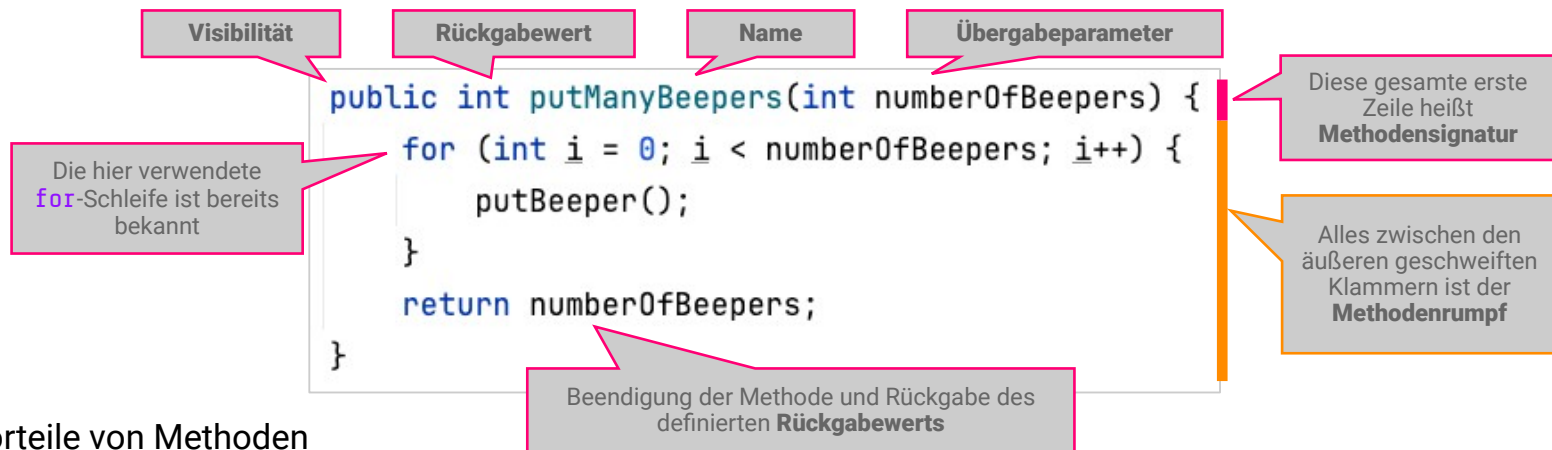
```
public int putManyBeeperes(int numberOfBeeperes) {  
    // Hier kommt Code hin  
}
```

Methodenrumpf

Der **Methodenrumpf** befindet sich zwischen den beiden äußersten geschweiften Klammern einer Methode. In ihm steht, was die Methode machen soll, was also ihre Funktionalität ist. Alle bisher behandelten syntaktischen Bausteine dürfen in einer Methode verwendet werden:

- **Variablen** dürfen deklariert und ihnen können Werte zugewiesen werden
- **Arithmetische Operationen** können ausgeführt werden
- **Verzweigungen und Schleifen** bestimmen den Ablauf einer Methode
- **Andere Methoden** dürfen aufgerufen werden

Aufbau einer Methode im Überblick



Vorteile von Methoden

- **Strukturierung** Durch die Verwendung von Methoden kann der Quellcode besser strukturiert werden, was dessen Komplexität reduziert und Verständnis erleichtert.
- **Weniger Redundanz** Wenn häufig benötigte Code-Teile in Methoden ausgelagert werden, müssen diese Code-Teile nicht immer wieder kopiert werden.

Fortgeschrittene Methoden-Konzepte

Parameter

In ihrer einfachsten Form haben Methoden weder Übergabeparameter noch einen Rückgabewert, z.B.:

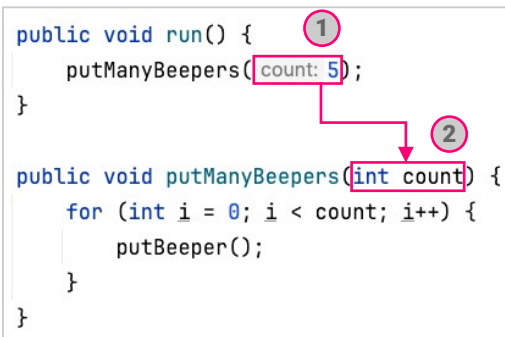
```
public void goToNextWall() {  
    while (frontIsClear()) {  
        move();  
    }  
}
```

Optional kann die aufrufende Methode der aufgerufenen Methode einen oder mehrere Variablen übergeben (**Übergabeparameter**) und damit das Verhalten der aufgerufenen Methode beeinflussen.

Hierfür sind zwei Dinge erforderlich:

- 1 Die aufrufende Methode muss den Übergabeparameter übergeben.
- 2 Die aufgerufene Methode muss in Ihrer Signatur den Übergabeparameter definieren.

```
public void run() {  
    putManyBeepers(count: 5);  
}  
  
public void putManyBeepers(int count) {  
    for (int i = 0; i < count; i++) {  
        putBeeper();  
    }  
}
```



```
public void run() {  
    fillAreaWithBeepers(columns: 5, rows: 6);  
}  
  
public void fillAreaWithBeepers(int columns, int rows) {  
    // Code zum Befüllen einer Fläche mit BEEPern.  
    // Die Größe der gewünschten Fläche wurde  
    // mittels zweier Parameter übergeben.  
}
```

Rückgabewerte

Eine Methode kann nach ihrer Ausführung entweder nichts zurückgeben oder genau einen Wert des in der Signatur angegebenen Typs. Wenn sie nichts zurückgibt, wird dies durch das Keyword `void` angegeben.

Soll etwas an die aufrufende Methode zurückgeliefert werden, sind hierfür zwei Dinge notwendig:

- 1 Der Typ des zurückgegebenen Werts muss in der Methodensignatur festgelegt werden.
- 2 Die Methode muss mittels `return` einen Wert des festgelegten Typs zurückliefern.

```
public void run() {  
    int b = getNumberOfBeepersHere();  
    System.out.println("Beepers: " + b);  
}  
  
1 int getNumberOfBeepersHere() {  
    int beepers = 0;  
    while (beeperIsPresent()) {  
        pickBeeper();  
        beepers++;  
    }  
    return beepers; 2  
}
```

```
public void run() {  
    boolean b = facingWestOrEast();  
    System.out.println("Is looking west or east: " + b);  
}  
  
1 boolean facingWestOrEast() {  
    return facingWest() || facingEast(); 2  
}
```

Wenn in der Signatur festgelegt ist, dass eine Methode einen Wert zurück liefert (d.h. es ist nicht `void` definiert), muss sie in jedem Fall exakt einen Wert dieses Typs oder einen **Fehler** zurückliefern (zu **Fehlern/Exceptions** mehr in Kapitel 3).

Methoden haben häufig sowohl Übergabeparameter als auch Rückgabewerte, die beliebig miteinander kombiniert werden können.

Call-by-Value

Die Übergabe von Parametern an Methoden erfolgt in Java nach dem **Call-by-Value-Prinzip**, d.h. der Methode wird immer eine Kopie des übergebenen Wertes bereitgestellt.

Das funktioniert nicht:

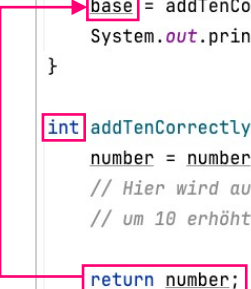
```
public void runWrongly() {
    int base = 5;
    addTenWrongly(base);
    System.out.println(base); // immer noch 5
}

void addTenWrongly(int number) {
    number = number + 10;
    // Zwar wurde hier number tatsächlich um 10 erhöht,
    // aber weil number nur eine separate Kopie ist,
    // bleibt der Wert in runWrongly unverändert bei 5.
}
```

Das funktioniert:

```
public void runCorrectly() {
    int base = 5;
    base = addTenCorrectly(base);
    System.out.println(base); // jetzt 15
}

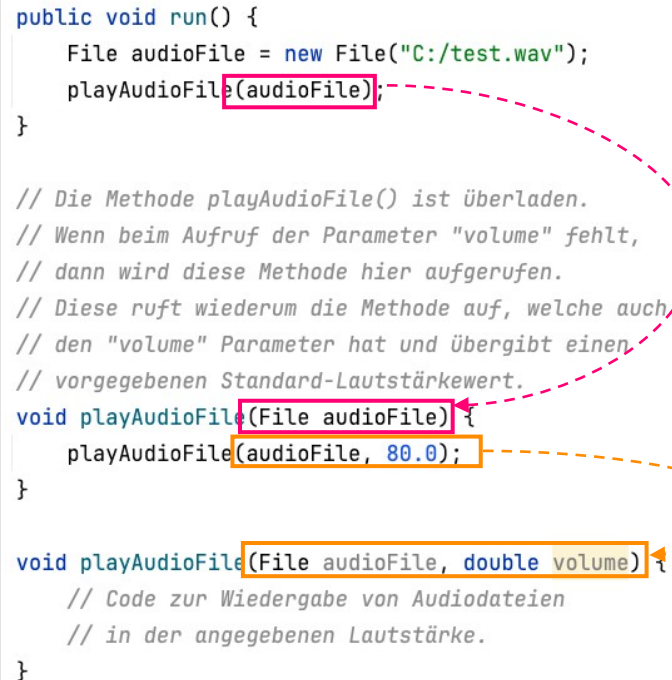
int addTenCorrectly(int number) {
    number = number + 10;
    // Hier wird auch nur die Kopie
    // um 10 erhöht
    return number;
    // Allerdings wird der neue Wert der Kopie
    // an runCorrectly() zurückgegeben
    // wo der neue Wert der dortigen
    // Variable "base" zugewiesen wird.
}
```

A pink arrow originates from the 'return number;' line in the 'addTenCorrectly' method and points to the 'base = addTenCorrectly(base);' line in the 'runCorrectly' method, illustrating how the returned value is assigned back to the original variable.

Überladene Methoden

Es ist möglich, mehrere Methoden zu definieren, welche zwar denselben Namen haben, aber sich in der Anzahl, Reihenfolge oder Art der Übergabeparameter unterscheiden. Man spricht dabei von einem **Überladen** der Methoden (**Method Overloading**):

```
public void run() {  
    File audioFile = new File("C:/test.wav");  
    playAudioFile(audioFile);  
}  
  
// Die Methode playAudioFile() ist überladen.  
// Wenn beim Aufruf der Parameter "volume" fehlt,  
// dann wird diese Methode hier aufgerufen.  
// Diese ruft wiederum die Methode auf, welche auch,  
// den "volume" Parameter hat und übergibt einen  
// vorgegebenen Standard-Lautstärkewert.  
void playAudioFile(File audioFile) {  
    playAudioFile(audioFile, 80.0);  
}  
  
void playAudioFile(File audioFile, double volume) {  
    // Code zur Wiedergabe von Audiodateien  
    // in der angegebenen Lautstärke.  
}
```



Die Main-Methode

Java-Programme können aus einer Vielzahl von Methoden bestehen, aber beim Start eines Programms wird immer zuerst die **Main-Methode** aufgerufen.

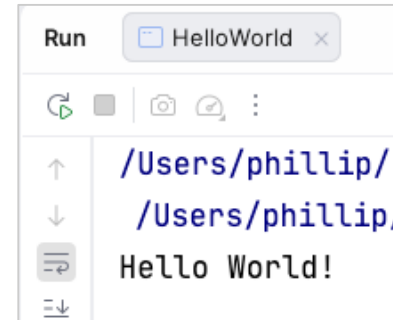
Sie ist der Einstiegspunkt für den gesamten Programmfluss.

Auch beim Start von Karel JRobot wird immer zuerst die Methode **Karel.main()** aufgerufen. Erst später wird der Code in der **run()** Methode ausgeführt.

Um von Grund auf ein neues Java-Programm zu entwickeln, muss also eine neue Klasse angelegt und mit einer Main-Methode versehen werden – z.B.:



```
1 public class HelloWorld {  
2  
3     public static void main(String[] args) {  
4         System.out.println("Hello World!");  
5     }  
6 }
```



```
Run HelloWorld  
↑ /Users/phillip/  
↓ /Users/phillip/  
Hello World!
```

Wenn beim Start des Programms Parameter über die **Kommandozeile** übergeben wurden, dann werden diese in dem Methoden-Parameter **args** hinterlegt (Es handelt sich dabei um ein **String-Array** – dazu später mehr).

Klassen Und Objekte

Klassen

Mittels Methoden lässt sich wunderbar definieren, welche Arbeitsschritte ein Programm durchführen soll (z.B. `feedAllZooAnimals()`). Oft werden Programme aber so groß und komplex (tausende Zeilen Code), dass Methoden allein zur Strukturierung nicht mehr ausreichen.

Bei der Objektorientierten Programmierung wird deshalb der Code (z.B. Methoden und Variablen) auf mehrere getrennte **Klassen** verteilt. Mit Klassen lässt sich definieren, welche Akteure im Programm eine Rolle spielen (z.B. `ZooKeeper`, `Elephant`, `Duck`). Jeder Akteur kann verschiedene Methoden haben.

In Java wird üblicherweise jede Klasse in einer eigenen ***.java** Datei definiert. Es ist aber auch möglich, in einer ***.java** Datei mehrere Klassen zu definieren.

Die Klasse `Duck` (in der Datei `Duck.java`) könnte z.B. folgendermaßen definiert sein:
Klassen sind Baupläne, die beschreiben:

```
// Enten haben einen Namen und ein Alter.  
// Sie können mit den Flügeln flattern und ihren Namen quaken.  
public class Duck {  
  
    // Variablen, welche für jedes Objekt vom Typ Duck getrennt gespeichert werden.  
    // Die vorgegebenen Standardwerte sollten geändert werden.  
    public String name = "?";  
    public int age = 0;  
  
    // Methoden, welche jedes Objekt vom Type Duck anbietet  
    public void flapWings(int flapCount) {  
        for (int i = 0; i < flapCount; i++) {  
            System.out.println("Flap");  
        }  
    }  
  
    public void sayName() {  
        System.out.println("Hi, my (quak) name is: " + name);  
    }  
}
```

- was ein Akteur *IST*, d.h. welchen Typ der Akteur hat
- was ein Akteur *HAT*, d.h. welche Attribute (Variablen/Konstanten) er besitzt (z.B. `name`, `age`)
- was ein Akteur *KANN*, d.h. welche Methoden er besitzt (z.B. `fly()` und `eat()`)

Objekte

Während Klassen Baupläne für die Akteure eines Programms sind, sind **Objekte** die Akteure selbst. Neue Objekte (auch **Instanzen** genannt) können mittels des **new** Operators erzeugt werden; so auch neue Instanzen des Typen **Duck**:

```
// Erstellen von zwei getrennten Objekten
// vom Typ Duck mittels des Keywords "new":
Duck freddy = new Duck();
Duck tina = new Duck();
```

Es existieren zwei voneinander unabhängige **Objekte (Instanzen)** vom Typ **Duck**.

Auf der Grundlage jeder Klasse können beliebig viele Objekte erstellt werden. Die zu den Attributen gehörigen Werte werden für jedes Objekt getrennt gespeichert.

Zur Laufzeit speichert Java alle Objekte (nicht jedoch primitive Datentypen) in einem Speicherbereich, der **Heap** genannt wird (dazu mehr in Kapitel 4).

Meistens möchte man nach der Erzeugung eines Objekts damit weiter arbeiten. Daher speichert man die **Referenz** (*quasi einen Link*) auf das erzeugte Objekt in einer eigenen Variable ab, welche dem Typ des Objekts entspricht.

Beispiel:

2. Die **Referenz** auf dieses Objekt wird in der Variable **freddy** vom Typ **Duck** gespeichert.

3. Über diese **Referenz** kann man jederzeit auf das Objekt und dessen **Attribute/Methoden** zugreifen.

```
// Erstellen eines Objektes
// vom Typ Duck mittels des Keywords "new":
Duck freddy = new Duck();

// Setzen von freddys Attributen
freddy.name = "Freddy";
freddy.age = 3;

// Aufruf von freddy Methoden
freddy.flapWings(3);
freddy.sayName();
```

1. Java erzeugt ein neues (**new**) Objekt vom Typ **Duck** und legt es im **Heap** ab.

Objektreferenzen

Anders als bei primitiven Datentypen, deren Variablen immer einen tatsächlichen Wert enthalten, speichern Variablen bei einem Referenztyp lediglich eine **Referenz** auf ein Objekt ab – wie der Name bereits sagt. Es ist möglich und üblich, dass mehrere Variablen auf dasselbe Objekt verweisen:

```
// Erstellen eines neuen Duck-Objektes.  
// Die Variable freddy referenziert auf dieses Objekt.  
Duck freddy = new Duck();  
// Die Variable auchFreddy referenziert auf exakt dasselbe Objekt  
Duck auchFreddy = freddy;
```

Im **Heap** existiert nur ein Objekt, aber zwei Variablen verweisen auf dieses.

Des Weiteren ist es auch möglich und üblich, dass eine Variable auf gar kein Objekt verweist. Die Variable enthält dann statt einer Objektreferenz den Wert **null**:

```
// Es existiert kein Duck-Objektes (kein new).  
// Die Variable freddy verweist auch auf kein Objekt (null).  
Duck freddy = null;
```

Das Keyword **this** funktioniert wie eine Konstante. Damit erhält ein Objekt die Referenz auf sich selbst.

Vergleichen von Variablen und Referenzen

Um zu prüfen ob zwei primitive Datentypen denselben Wert haben, kann man wie gewohnt beide Variablen mittels `==` vergleichen:

```
int a = 5;
int b = 5;
if (a == b) {
    System.out.println("a und b sind gleich");
}
```

Da Variablen bei Objekt-Datentypen nur Links auf Objekte speichern, prüft ein `==` bei Objekt-Variablen, ob diese auf die selbe Instanz verweisen. Hier verweisen a und b nicht auf dasselbe Objekt (es sind ja 2 separate **Ducks**).

```
Duck a = new Duck();
Duck b = new Duck();
if (a == b) {
    System.out.println("a und b referenzieren dasselbe Objekt");
} else {
    System.out.println("a und b referenzieren nicht dasselbe Objekt");
}
```

Möchte man prüfen ob zwei Objekte inhaltlich gleich sind, verwendet man die `equals`-Methode (hier sind beide Strings separat, *aber inhaltlich gleich*):

```
String a = "Freddy";
String b = "Freddy";
if (a.equals(b)) {
    System.out.println("a und b sind inhaltlich gleich");
}
```

Die `equals`-Methode gibt in folgenden Fällen `true` zurück:

- Beide Variablen verweisen auf dasselbe (eine) Objekt.
- Die Variablen verweisen auf 2 getrennte Objekte – doch diese sind inhaltlich gleich

Konstrukturen

Um sicherzustellen, dass bei der Erzeugung jedes neuen **Duck**-Objekts immer gleich **name** und **age** gesetzt werden, können wir die Klasse **Duck** um einen **Konstruktor** erweitern. Ein Konstruktor sieht sehr ähnlich zu einer Methode aus, allerdings hat er keinen Rückgabewert und sein Name ist gleich dem Klassennamen, in dem er steht:

Syntax:

```
public Duck(Parameter) {  
    // Konstruktor Code  
}
```

```
public class Duck {  
  
    // Variablen, welche für jedes Objekt vom Typ Duck getrennt gespeichert werden.  
    // Diese Variablen werden bei Verwendung des neuen Konstruktors einmalig gesetzt.  
    public String name;  
    public int age;  
  
    // Konstruktor:  
    // Um Duck-Objekte erzeugen zu können,  
    // müssen jetzt immer name und age angegeben werden.  
    public Duck(String name, int age) {  
        // Um zwischen der Instanzvariable "name" (oben definiert)  
        // und dem Konstruktor-Parameter (auch "name") unterscheiden zu können,  
        // adressieren wir die Variable dieser Instanz explizit mit "this".  
        this.name = name;  
        this.age = age;  
    }  
    //...
```

Standardmäßig besitzt jede Klasse einen Konstruktor ohne Übergabeparameter [z.B. `public Duck()`]. Sobald aber ein eigener Konstruktor geschrieben ist, ist der Standardkonstruktor nicht mehr verwendbar, d.h. es muss der neue verwendet werden:

```
// Erstellen eines Objektes  
// vom Typ "Duck"  
Duck freddy = new Duck();  
freddy.name = "Freddy";  
freddy.age = 2;
```

Expected 2 arguments but found 0
Create constructor ↵ More actions... ↵

```
// Erstellen eines Objektes  
// vom Typ "Duck"  
// mittels des neuen Konstruktors  
Duck freddy = new Duck("Freddy", 2);
```

Klassen können auch mehrere Konstrukturen haben, die sich aber in den Parameter-Typen oder der Parameter-Reihenfolge unterscheiden müssen (gleiche Regeln wie beim Methoden-Overloading).

Kapselung und Validierung

Aktuell ist der interne Zustand aller **Duck** Objekte von überall aus beliebig veränderbar. Jeder Code-Bestandteil kann z.B. dem **Duck** hinter der Variable **freddy** ein ungültiges Alter zuweisen:

```
// Das geht leider:  
Duck freddy = new Duck("", -42);  
  
// Das geht leider auch noch:  
freddy.name = "lol!";  
freddy.age = 1337;
```

Um den internen Zustand der **Duck** Objekte besser zu schützen, sollte im ersten Schritt sichergestellt werden, dass nur noch die Klasse **Duck** uneingeschränkt auf die Attribute **name** und **age** zugreifen kann (lesen/schreiben). Dies wird erreicht, indem statt des **Visibility-Keywods public** das Visibility-Keyword **private** verwendet wird (Visibility wird ausführlich in Kapitel 3 behandelt):

```
public class Duck {  
  
    // Zugriffsrechte schützen Variablen, welche für jedes Objekt vom Typ Duck getrennt gespeichert werden.  
    private String name;  
    private int age;
```

Andere Klassen (z.B. **Zoo**) können nun nicht mehr direkt auf die Attribute zugreifen. D.h. nun können andere Klassen diese Attribute weder direkt lesen noch verändern.

```
// Das geht nicht mehr  
// wegen private-visibility:  
freddy.name = "lol!";  
freddy.age = 1337;
```

'age' has private access in 'Duck'

Für das Funktionieren des Zoos ist es jedoch notwendig, dass andere Klassen auf die Attribute **name** und **age** der **Duck**-Klasse zugreifen können (lesen und verändern). Dies soll allerdings nur in genau festgelegten Grenzen möglich sein.

Hierfür bietet **Duck** zwei **Setter-Methoden** an, über welche die Attribute **name** und **age** verändert (d.h. geschrieben, ersetzt) werden können.

Der Weg des Übergabeparameters (Zoo → Duck-Konstruktor → Setter → Attribut) ist unten am Beispiel von **age** markiert.

```
public class Zoo {  
    static void main() {  
  
        Duck freddy = new Duck("Freddy", 3);  
  
        freddy.sayName();  
    }  
}
```

Start

```
public class Duck {  
  
    // Deklaration der Attribute von Duck  
    // private, d.h. von anderen Klassen nicht les-/schreibbar  
    private String name;  
    private int age;  
  
    // Konstruktor (wird bei "new" aufgerufen)  
    public Duck(String name, int age) {  
        // Um zwischen der Instanzvariable "name" (oben definiert)  
        // und dem Konstruktor-Parameter (auch "name") unterscheiden zu können,  
        // adressieren wir die Variable dieser Instanz explizit mit "this".  
        setName(name);  
        setAge(age);  
    }  
  
    // Setter-Methoden ermöglichen anderen Klassen  
    // das Setzen der Attribute von Duck (name und age):  
    public void setAge(int age) {  
        this.age = age;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

Ziel

Attribut age
des **Duck**-Objekts (orange).
Um Verwechslungen mit dem gleichnamigen
Übergabeparameter (magenta) zu
vermeiden, wird das **Duck**-Objekt mittels
this angesprochen.

Methoden-Übergabeparameter age

Die Setter-Methoden werden nun so verbessert, sodass die Attribute **name** bzw. **age** von **Duck** (ganz oben) nur verändert werden, falls die vorgeschaltete **Validierung** (Prüfung) der Übergabeparameter (*magenta Markierung*) erfolgreich war. Andernfalls wird die Variable nicht verändert, sondern lediglich eine **Fehlermeldung** angezeigt:

```
// Setter-Methoden ermöglichen anderen Klassen
// das Setzen der Attribute von Duck (name und age):
public void setAge(int age) {
    // Prüfung des Methoden-Übergabeparameters "age"
    if (age > 0 && age < 30) {
        // Überschreibe das Klassen-Attribut "age" (alt, links)
        // mit dem Wert des Übergabeparameters "age" (neu, rechts)
        this.age = age;
    } else {
        System.err.println("Invalid age: " + age);
    }
}

public void setName(String name) {
    // Prüfung des Methoden-Übergabeparameters "name".
    if (name != null && name.length() > 1 && name.length() < 100) {
        this.name = name;
    } else {
        System.err.println("Invalid name: " + name);
    }
}
```

Durch die **Kapselung** der Variablen und die **Wert-Validierung** in den **Setter-Methoden** ist nun der interne Zustand der **Duck**-Objekte gesichert:

```
Duck freddy = new Duck("F", 300);

Invalid name: F
Invalid age: 300
```

```
freddy.setName("Dasisteinlangernamedasisteinlangernamedasisteinlangernamedasisteinlangernamedasisteinlangernamedasiste");
freddy.setAge(9000);

Invalid name: Dasisteinlangernamedasisteinlangernamedasisteinlangernamedasisteinlangernamedasisteinlangernamedasiste
Invalid age: 9000
```


Neben den Setter-Methoden bieten Klassen häufig auch noch **Getter-Methoden** an:

```
public class Duck {  
  
    // Zugriffsgeschützte Variablen, welche für jedes Objekt vo Typ Duck getrennt gespeichert werden.  
    private String name;  
    private int age;  
  
    // Liefert den Namen der Ente zurück  
    public String getName() {  
        return name;  
    }  
  
    // Liefert das Alter der Ente zurück  
    public int getAge() {  
        return age;  
    }  
  
    //...
```

Die Getter- und Setter-Methoden werden als **Akzessoren (Accessor-Methods)** bezeichnet, da sie einen Zugriff auf die jeweiligen Attribute bieten.

toString

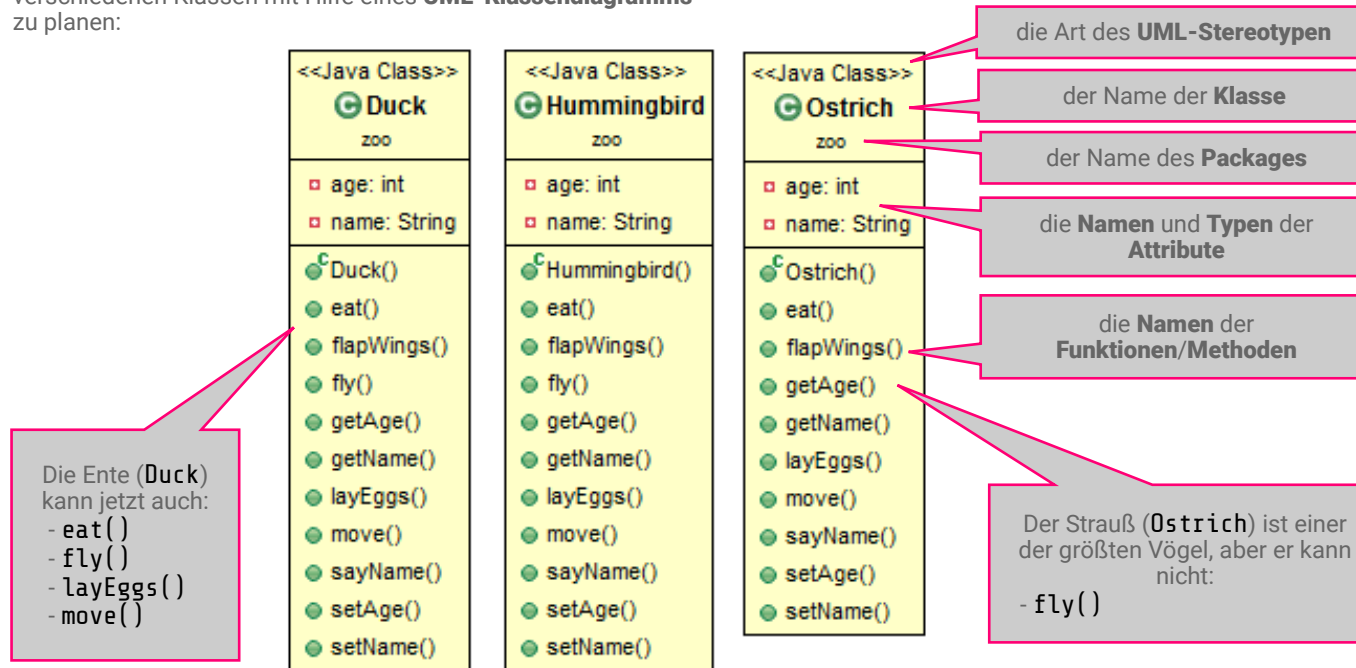
Damit man zur Laufzeit den Typ und aktuellen Zustand eines Objektes erfahren kann, bietet es sich an, für jede eigene Klasse die **toString()** Methode zu definieren:

```
public class Duck {  
  
    public String toString() {  
        return "I am " + getName() + " the Duck and my age is " + getAge() + ".";  
    }  
}
```

Vererbung

Es kann nun eine Menge abgesicherter Enten erzeugt werden, aber für einen richtigen Zoo werden ein paar weitere verschiedene Klassen (*Tierarten*) benötigt.

Bevor man mit dem Programmieren anfängt, ist es oft ratsam die verschiedenen Klassen mit Hilfe eines **UML-Klassendiagramms** zu planen:



Wenn man `Duck.java` zwei mal kopiert, den Konstruktor umbenennt und einmal `fly()` herauslöscht, erhält man oben gezeigtes Ergebnis.

Wenn sich aber irgendwann das Fressverhalten aller Tiere ändern soll [`eat()`], muss dreimal der gleiche Code geändert werden. Hier gibt es eine bessere Lösung!

In der Biologie werden Organismen entsprechend ähnlicher Eigenschaften hierarchisch klassifiziert ([Biosystematik](#)), z.B. für die Stockente:

<i>Unterstamm</i>	→ <i>Klasse</i>	→ <i>Ordnung</i>	→ <i>Familie</i>	→ <i>Tribus</i>	→ <i>Gattung</i>	→ <i>Art</i>
Wirbeltiere	→ Vögel	→ Gänsevögel	→ Entenvögel	→ Schwimmenten	→ Eigentliche Enten	→ Stockente

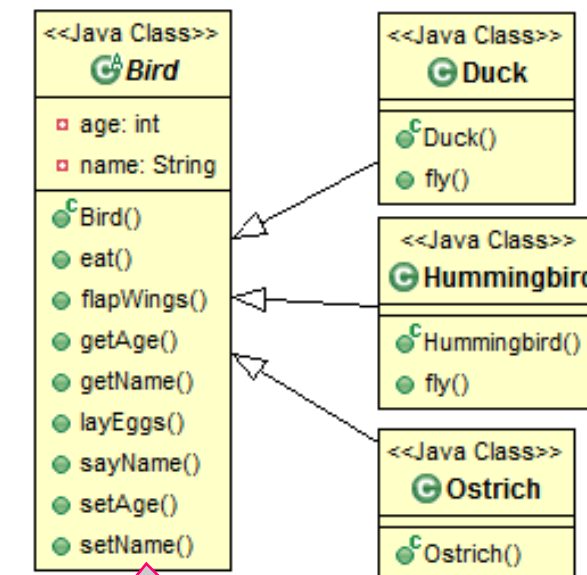
Mitglieder der gleichen Gruppierung haben gleiche Eigenschaften. Alle Vögel etwa:

- haben Flügel (aber nicht alle können auch fliegen)
- pflanzen sich durch Eiablage fort

Auch in Java können Hierarchien von Klassen gebildet werden. Es kann beispielsweise eine **Mutterklasse** namens **Bird** definiert werden, die wiederum die Methoden **flapWings()**, **layEggs()**, **setAge()**, etc. enthält.

Die Unterklassen **Duck**, **Hummingbird** und **Ostrich** können von dieser Mutterklasse **erben**. Damit stehen den Unterklassen automatisch die Funktionalitäten der Mutterklasse **Bird** zur Verfügung.

In den Unterklassen müssen die Methoden der Mutterklasse folglich nicht noch einmal implementiert werden. Dadurch reduziert sich die Gesamtmenge des Codes erheblich.



All diese Methoden sind jetzt nur noch einmal (statt dreimal) definiert.

In Java wird eine Vererbung zwischen Klassen mit folgender Syntax angezeigt:

Syntax:

```
class ChildClass extends ParentClass
```

```
// Die Unterklasse "Duck" erbt von der Mutterklasse "Bird".
// Dadurch ist der Quellcode der Duck-Klasse viel kleiner geworden.
public class Duck extends Bird {

    public Duck(String name, int age) {
        setName(name);
        setAge(age);
    }

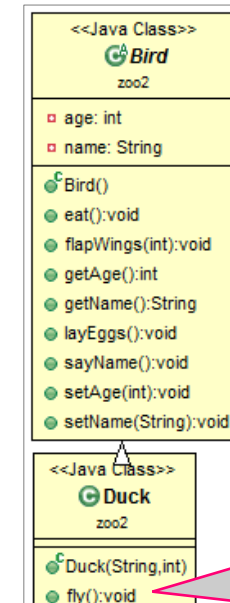
    // Duck definiert eine eigene fly() Methode,
    // weil die Mutterklasse keine bereit stellt.
    public void fly() {
        // Duck kann aber auf die von der Mutterklasse
        // bereitgestellte Methode flapWings zugreifen.
        flapWings(7);
        System.out.println("I am flying!");
    }
}
```

Im obigen Beispiel ist **Duck** die Unterklasse/Kindklasse und **Bird** die Oberklasse/Mutterklasse.

Unterklassen können (nahezu) beliebig zusätzliche Variablen und Methoden definieren, welche nicht in der Mutterklasse definiert sind.

Unterklassen können Methoden aufrufen, welche bereits in der Mutterklasse definiert sind (*Markierung in magenta*) – außer wenn diese Methoden **private** sind.

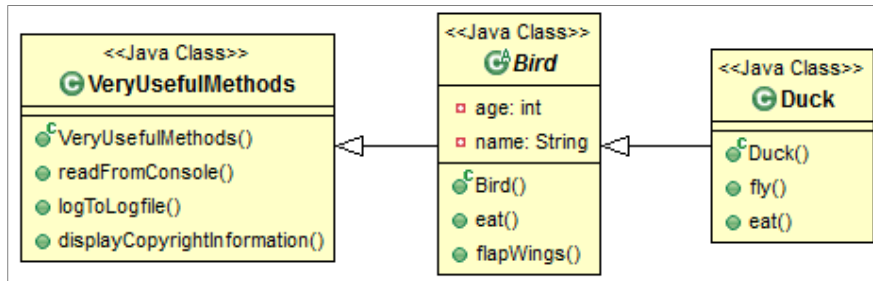
Auch auf Variablen, die in der Mutterklasse definiert sind, kann eine Unterklasse zugreifen – außer diese Variablen sind als **private** markiert.



Duck
implementiert eine
zusätzliche **fly()**
Methode.

Im Gegensatz zu manch anderen objektorientierten Programmiersprachen unterstützt Java keine **Mehrfachvererbung**, d.h. eine Klasse kann maximal von einer Mutterklasse erben. Die Mutterklasse kann aber ihrerseits von einer anderen Mutterklasse erben.

Dass **Unterklassen** automatisch die Funktionalitäten der **Oberklasse** erben ist praktisch. Aus Bequemlichkeit könnte man also auf folgende Idee kommen:



Das wäre zwar technisch machbar, widerspricht aber der geltenden Konvention:

	<i>EIN</i>	Unterklasse	<i>IST EIN</i>	Oberklasse
z.B.:	<i>EIN</i>	Duck	<i>IST EIN</i>	Bird
	<i>EIN</i>	Bird	<i>IST EIN</i>	Animal
	<i>EIN</i>	Duck	<i>IST EIN</i>	Animal
aber:	<i>EIN</i>	Bird	<i>IST KEIN</i>	VeryUsefulMethods

Das Keyword **extends** ist gleich zu setzen mit „ist ein“ sowie „erbt von“.

Wenn eine Unterklasse eine Methode definiert und die Mutterklasse bereits eine Methode mit derselben Signatur (Name und Parameter) enthält, dann **überschreibt** die Methode der Unterklasse die Methode der Mutterklasse. Dies wird als Method **Overriding** bezeichnet (nicht zu verwechseln mit Overloading).

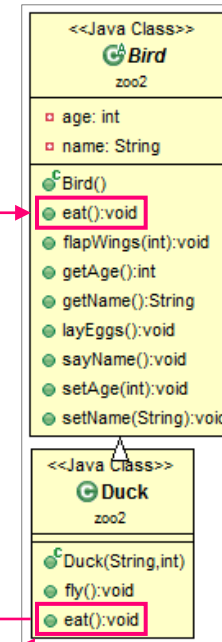
Bei einem Aufruf von `Duck.eat()` wird ausschließlich die in `Duck` definierte Methode verwendet (nicht die von `Bird`).

```
// Die Unterklasse "Duck" erbt von der Mutterklasse "Bird".
// Dadurch ist der Quellcode der Duck-Klasse viel kleiner geworden.
public class Duck extends Bird {

    public Duck(String name, int age) {
        setName(name);
        setAge(age);
    }

    // Duck definiert eine eigene fly() Methode,
    // weil die Mutterklasse keine bereit stellt.
    public void fly() {
        // Duck kann aber auf die von der Mutterklasse
        // bereitgestellte Methode flapWings zugreifen.
        flapWings(7);
        System.out.println("I am flying!");
    }

    // Duck kann auch die von der Mutterklasse definierte
    // Methode eat() überschreiben.
    // Wenn nun die eat() Methode eines Duck-Objektes aufgerufen wird,
    // dann wird anstatt der von der Bird-Klasse definierte Methode
    // die hier definierte Methode aufgerufen.
    public void eat() {
        System.out.println("Hmm delicious, quak!");
    }
}
```



Duck überschreibt die von Bird definierte `eat()` Methode.

Bei der Beschreibung von Objekten sind vier Hauptaspekte maßgeblich, welche auch in Klassendiagrammen dargestellt werden können:

Was ist das für ein Objekt?

Dies ergibt sich aus der Klasse (und ggf. den Mutterklassen) sowie den implementierten **Interfaces** (dazu später mehr), z.B.:

EIN Duck IST EIN Bird $\wedge$ *EIN Bird IST EIN Animal* $\rightarrow$ *EIN Duck IST EIN Animal*

logisches **UND**

Implikation (Kettenschluss)

Was kann das Objekt?

Dies ergibt sich aus den Methoden, die das Objekt anbietet. Das wiederum ergibt sich direkt aus der Klasse des Objekts (und ggf. den Mutterklassen/Schnittstellen) sowie den implementierten Interfaces, z.B.:

Die Klasse **Duck** implementiert die **fly()** Methode, d.h. alle Enten können per se fliegen.

Was hat das Objekt?

Aus der Klasse ergibt sich, welche Attribute das Objekt besitzt, z.B.:

Ein **Duck** hat immer einen **name** und ein **age**.

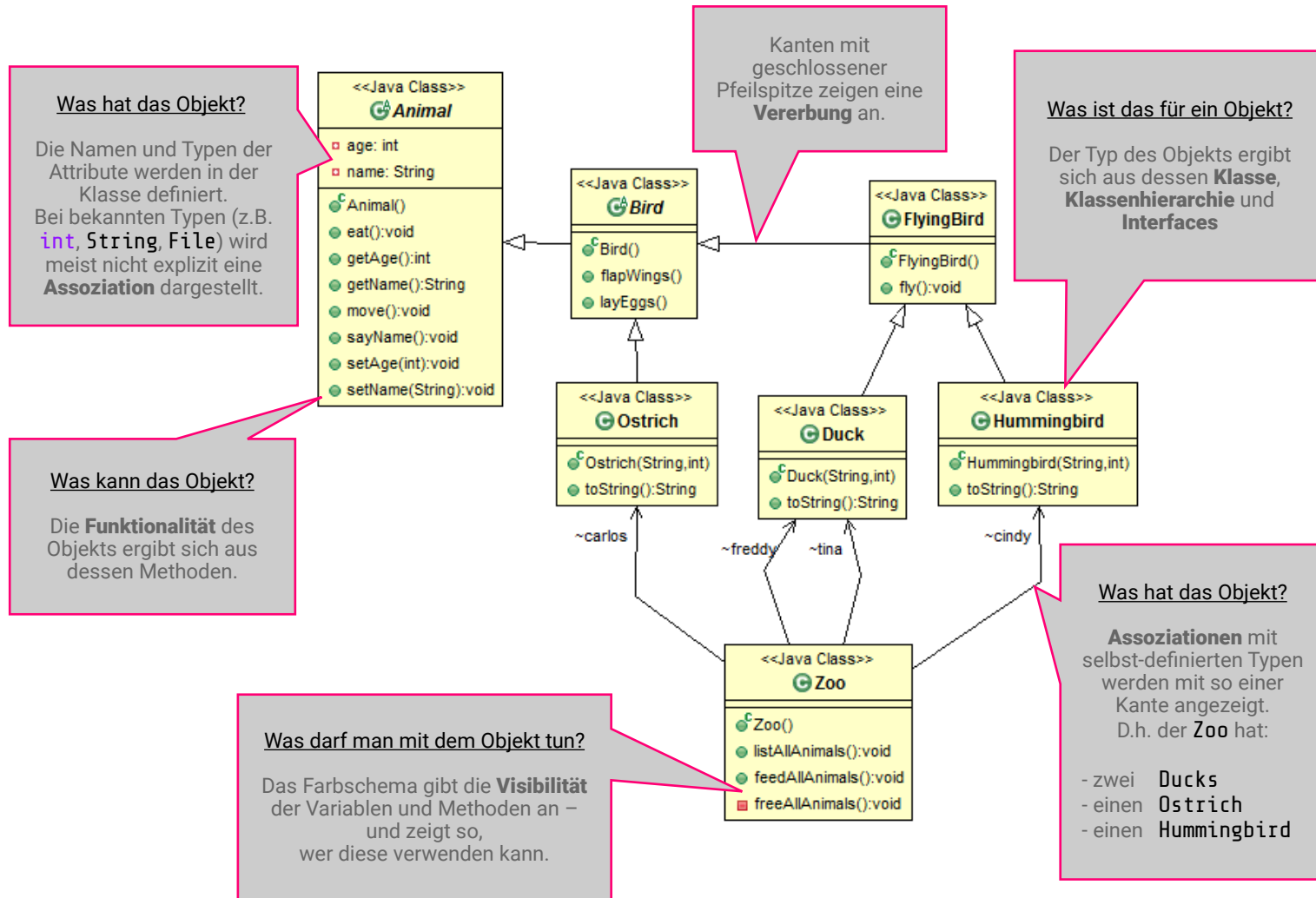
Wenn eine Klasse (z.B. **Zoo**) eine andere Klasse (z.B. **Duck**) referenziert (z.B. **Zoo** hat eine Variable vom Typ **Duck**), dann bezeichnet man diese Beziehung als **Assoziation**.

Eine Sonderform der Assoziation ist die **Komposition**, bei der die Existenz der Unterbestandteile (z.B. **Room**) direkt abhängig ist von der Existenz des Ganzen (z.B. **House**) ist.

Was darf man mit dem Objekt tun?

Einerseits ergibt sich das aus der Visibilität der Variablen und Methoden. Andererseits kann der Zugriff noch feiner geregelt werden, z.B. durch Validationen in Akzessor-Methoden (siehe die Setter-Methoden von **Duck**).

Klassendiagramme des Zoos



Statische Variablen und lokale Variablen

Bisher wurde für jedes einzelne erzeugte Duck Objekt (für jede Instanz) ein eigener Namen und ein eigenes Alter festlegen. Für n Ducks wurden also n Werte für je **age** und **name** gespeichert. Die dafür eingesetzten Variablen heißen **Instanzvariablen**.

Mittels **Klassenvariablen** ist es möglich, eine Variable auf Klassen-Ebene zu speichern. Der Variablen-Wert wird hierbei nur ein einziges mal im Arbeitsspeicher abgelegt, egal wie viele n Objekte von der Klasse erstellt werden. Alle Objekte der Klasse greifen auf diesen einen, instanz-übergreifenden Wert zu. Klassenvariablen werden mittels des Keywords **static** deklariert:

Syntax:

```
static MyType myName;
```

```
public class Duck extends Bird {  
  
    // Klassenvariable (statisch)  
    // Alle Objekte/Instanzen der Klasse Duck teilen sich diesen einen Wert  
    public final static String SPECIES_DESCRIPTION = "It walks, swims and quaks like a duck.";
```

Instanzvariablen und Klassenvariablen werden immer außerhalb von Methoden deklariert. Variablen die innerhalb von Methodenrumpfen deklariert werden, heißen **Lokale Variablen**.

Variablen jeder Art (Instanzvariablen, Klassenvariablen, Lokale Variablen) können mittels des Keywords **final** in Konstanten umgewandelt werden.

Für Fortgeschrittene (nicht klausurrelevant): [Local-Variable Type Inference \(JEP 286\)](#).

Statische Methoden

Es gibt auch Fälle, in denen es sinnvoll ist, dass man Methoden einer Klasse aufrufen kann, ohne explizit mit **new** eine Instanz dieser Klasse zu erstellen. Hierfür können **statische Methoden** (Klassenmethoden) eingesetzt werden.

Beispielsweise kann man sich so die Beschreibung zu Enten ansehen, ohne explizit eine neue Ente mit eigenem Namen und Alter erzeugen zu müssen:

static wird vor allem aus Komfort-Gründen bei sogenannten **Helper-Methoden** eingesetzt, z.B.:

```
public class Duck extends Bird {  
  
    // Klassenvariable (statisch)  
    // Alle Objekte/Instanzen der Klasse Duck teilen sich diesen einen Wert  
    public final static String SPECIES_DESCRIPTION = "It walks, swims and quaks like a duck.";  
  
    // Statische Methode  
    public static void printSpeciesDescription() {  
        System.out.println(SPECIES_DESCRIPTION);  
    }  
}
```

```
// Wir können die statische Methode aufrufen, ohne  
// ein neues Objekt zu erzeugen.  
Duck.printSpeciesDescription();
```

`System.out.println();`

`out` ist eine **statische Konstante** der Klasse `System`. Sie hat den Typ `PrintStream`.

`println()` ist eine Methode der Klasse `PrintStream`.

Kapitel 2 - Zusammenfassung

- Komplexe Programme können mittels **Methoden** strukturiert werden. Der Einstiegspunkt für Java-Programme ist die **Main-Methode**.
- Bei der Objektorientierten Programmierung wird der Code auf mehrere **Klassen** und Methoden verteilt. Eine Klasse ist ein Bauplan für einen Akteur des Programms, welcher dessen Attribute und Methoden definiert. Auf der Grundlage jeder Klasse können beliebig viele **Objekte (Instanzen)** erstellt werden.
- Mittels parametrisierter **Konstruktoren** können die Attribute eines Objekts bereits bei dessen Erzeugung gesetzt werden.
- **Kapselung** dient dem Schutz des inneren Zustands von Objekten vor unzulässigen Zugriffen von außen. Hierbei wird häufig die **Visibility** von **Attributen** eingeschränkt und der Zugriff über **Akzessor-Methoden** kontrolliert.
- Die Attribute und Methoden von Klassen sowie die Beziehungen der Klassen untereinander können in einem **UML Class Diagram** dargestellt werden.
- Mittels **Vererbung** können **Unterklassen** Attribute und Methoden ihrer **Überklassen** übernehmen bzw. auf diese zugreifen.
- Beim **Method-Overloading** haben mehrere Methoden denselben Namen aber unterscheiden sich hinsichtlich ihrer Übergabeparameter (unterschiedliche Signatur).
- Beim **Method-Overriding** ersetzt eine in der Kindklasse definierte Methode eine andere Methode die bereits mit gleicher Signatur in der der Überklasse definiert war.
- **Instanzvariablen** werden für jedes Objekt einer Klasse getrennt gespeichert.
- **Klassenvariablen** (**static**) werden für alle Objekte einer Klasse zentral gespeichert. **Statische Methoden** können verwendet werden ohne explizit ein Objekt erzeugen zu müssen.